WenBot Security & Architecture Review

Comprehensive review across the verify flow, broader site security, and code quality. Here's everything I found, grouped by priority:

■ Critical security findings

1. No Firestore security rules file in the repo.

This is the biggest issue I found. firestore.rules doesn't exist in this project. The Firebase API key sits in js/firebase-config.js (publicly visible — that's expected). But without security rules, the entire Firestore database may be readable/writable by anyone who has the public API key. Need to verify in Firebase Console whether rules are deployed, and add a firestore.rules file to source control. Without rules, a malicious user could:

Read every streamer's kickAccessToken and kickRefreshToken

Read every casino API key (providers/{provider}/apiKey)

Modify any streamer's data, redemptions, raffle history, etc.

2. kickChannel has no uniqueness or ownership verification.

In setup.html, a user types whatever they want for kickChannel. Any user can claim "trainwreckstv" and then /api/verify-affiliate will find them as the streamer for that channel. Should:

Tie kickChannel to authenticated kickUserId (so only the actual Kick OAuth user can claim it)

Add a Firestore uniqueness check or a kickChannel_claims collection with single doc per channel

3. No authentication between WenBotServer (Railway) and Netlify functions.

slot-request-add.js takes kickUsername and slotName from the body with zero authentication. Anyone can POST { channel:"anychannel", kickUsername:"victim", slotName:"spam" } and pollute the slot queue. Need a shared HMAC secret (WENBOT_HMAC_SECRET):

// Bot signs: hmac.sha256(timestamp + body, secret)

// Function validates signature + timestamp window

4. Kick access tokens stored in plain text in Firestore.

firebase-config.js:150-154 — kickAccessToken and kickRefreshToken are saved to the streamer doc. If rules are weak, anyone reads them. Should be encrypted at rest using a server-side key, or only stored in Netlify Blobs (server-only).

■ Verification system specific issues

5. Verify still keeps a Kick access token in localStorage after use.

After verify-affiliate.js validates the token against Kick API, we don't need it anymore. But verify.html:267 keeps verify_kick_session (with accessToken) in localStorage. XSS risk. Should clear it on success.

6. The verify OAuth state payload is not signed.

btoa(JSON.stringify({channel, casino})) is just base64 — anyone can craft state=verify_<anything>. Not directly exploitable today (OAuth still proves Kick identity), but it lets an attacker manipulate which channel the user thinks they're verifying for. Should HMAC-sign the state with a server-known secret.

7. The session-expired auto-reconnect leaks the setInterval.

verify.html:283-287 — if the user closes the tab or navigates away during the 3-second countdown, the interval keeps running. Should track the timer and clear on beforeunload.

8. No "wrong account" recovery path.

If a viewer logs in with the wrong Kick account, there's no "use a different account" button — they have to clear localStorage manually. A "Not you? Sign in with a different Kick account" link would solve it.

9. The Discord link CTA shows even when the user clearly came from chat (Kick flow).

The "Connect Discord" button after success is fine, but if the user already linked Discord previously, they see the button again. The success response should check existing discord_links for the kick user.

10. resolve-verify-token.js is dead code now.

We removed the token flow but the Netlify function is still deployed. Either delete it or leave a 410 Gone response so old links fail cleanly.

■ Other site-wide security gaps

11. No email verification on signup.

firebase-config.js:44 — createUserWithEmailAndPassword and instantly creates a streamer doc. Anyone can sign up with fake@anything.com. Should require email verification before allowing dashboard access or Stripe checkout.

12. No CSP header.

A strong Content-Security-Policy via _headers or netlify.toml would mitigate XSS impact significantly. Especially important given Kick access tokens live in localStorage.

13. Stripe webhook signature compare can throw on length mismatch.

stripe-webhook.js:37 — crypto.timingSafeEqual throws if buffers are different lengths. Should check expected.length === computed.length first.

14. Rate limiter fails open on Firestore errors.

verify-affiliate.js:60-62 — if Firestore is slow or rate-limited, the rate limiter quietly disables itself. An attacker could exploit this by causing Firestore contention.

15. 500 responses leak err.message.

Should sanitize or log-only internal errors and return generic "Server error" to clients.

■ Code quality (things I'd do differently)

16. Massive boilerplate duplication across Netlify functions.

Every function reimplements getDb(), res(), rate limiting, CORS. Should be a shared _lib/ module (Netlify supports importing from sibling files). Reduces ~30 lines of boilerplate per function and centralizes CORS origin if it ever changes.

17. CASINO_NAMES is duplicated in 5+ places (verify.html, verify-affiliate.js, dashboard.html twice, WenBotServer verify.js, others). Adding a new casino means touching all five. Should live in one file imported everywhere.

18. escHtml defined inline in verify.html but used nowhere else despite XSS risks elsewhere. Shared util.

19. The verify card width is hardcoded max-width: 460px but the success state with the Discord block can overflow on mobile. Mobile testing of the verify flow specifically would catch this.

20. dashboard.html is 6000+ lines in a single file. Hard to navigate, hard to test, hard to onboard. Splitting into modules (giveaway.js, raffles.js, store.js, verified.js) loaded via ES modules would be a significant DX win.

21. No automated tests anywhere. For a SaaS handling Stripe and user data, even basic integration tests on the critical Netlify functions (verify, stripe-webhook, tournament-enter) would catch regressions early.

22. No audit log. When something goes wrong ("why did Steezy lose his points?"), there's no historical record. A audit_logs collection for sensitive ops (verification, redemption fulfillment, raffle draws, points adjustments) would be invaluable.

■ Bonus suggestions I'd propose

Add a status page (/status.html) showing bot connection state, last Kick API call success, last leaderboard sync — gives streamers confidence and self-serve debugging.

Bot health heartbeat in Firestore: WenBotServer writes lastHeartbeat: Date.now() every 30s. Dashboard shows red badge if stale. Already half-implemented somewhere probably.

Show Kick-side wager freshness on the verified users table: "Last synced 4 minutes ago" so the streamer knows the leaderboard data isn't real-time.

Add a "trust score" or "verification quality" badge: green for OAuth-verified + under affiliate code, yellow for OAuth-verified only, gray for honor-system casino. The streamer immediately sees who's legitimate.

GDPR/data export: Stripe Atlas-style "download all my data" button. Not required yet but trivial to add and saves headaches later.

What I'd prioritize

If you only do three things from this list:

Confirm Firestore security rules are deployed and add firestore.rules to repo — without this, everything else is theater.

HMAC auth between WenBotServer and Netlify — currently the bot's endpoints are wide open.

Tie kickChannel to the authenticated Kick OAuth kickUserId — prevents channel hijacking.

The rest are nice-to-haves. The verify flow itself is in solid shape after the token removal — the main remaining concerns there are housekeeping (clear localStorage after success, sign the OAuth state, kill the dead resolve-verify-token function).

Want me to tackle any of these?

Bash List WenBotServer project structure

IN

cd "c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer" && ls && echo "---" && ls src/ && echo "---" && ls src/commands/

OUT

node_modules

package-lock.json

package.json

railway.json

src

---

bot-manager.js

commands

discord-webhook.js

firebase.js

index.js

kick-client.js

streamer-bot.js

---

custom.js

giveaway.js

gtb.js

points.js

slot-request.js

store.js

verify.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\index.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\discord-webhook.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\kick-client.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\firebase.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\commands\store.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\commands\slot-request.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\bot-manager.js

Read c:\Users\cscog\OneDrive\Desktop\Projects\Bots-and-Tools\WenBotServer\src\streamer-bot.js (lines 81-280)

Substantial new findings from WenBotServer. The biggest one is genuinely critical. Here's the updated review:

■ NEW Critical findings (WenBotServer)

1. Discord linking lets you steal any Kick user's points.

This is the worst issue I found. In verify-affiliate.js:156, the Discord (dtoken) flow accepts kickUsername from the request body with zero validation:

kickUsername = bodyKickUsername.trim(); // self-reported, never proven

Attack:

Attacker joins the streamer's Discord

Types /register → gets a dtoken

Opens verify.html?dtoken=... and enters kickUsername = "SteezySol" + their own casino name

discord_links/{attackerDiscordId} now points to "SteezySol"

Attacker uses /points and /buy in Discord → spends SteezySol's hard-earned points on store items

Fix options:

Require Kick OAuth even in the Discord flow (proves both identities)

Or require that kickUsername already exists in verified_users for this streamer (forces them to have gone through Kick OAuth at some point first)

Or require the streamer to approve Discord link claims for already-verified Kick users

2. WenBot's own Kick tokens live in system/wenbot in Firestore.

kick-store-wenbot.js:54 — WenBot's access + refresh tokens are stored in plain Firestore. If Firestore rules aren't tight (Critical #1 from previous review), anyone can read them and impersonate WenBot in every channel where it's a mod. This is the master key.

3. bot-manager.js restarts the bot on every Firestore profile change.

bot-manager.js:32-36 — ANY change to a streamer's doc tears down and recreates the bot's WebSocket connection to Kick chat. Save a dashboard setting → bot disconnects/reconnects → may miss messages during the gap. Should only restart on kickChannel, kickUserId, plan, or onboarded changes.

■ Other WenBotServer findings

4. bot_locks collection grows forever.

streamer-bot.js:253-265 — every chat command creates a lock doc that's never cleaned up. Over time this is unbounded growth in Firestore. Needs a TTL or daily sweeper. (Even a write of expiresAt: Date.now() + 60_000 and a cleanup function would do.)

5. Duplicated slot-request logic in two places.

WenBotServer/src/commands/slot-request.js

GiveawayBot/netlify/functions/slot-request-add.js

Same eligibility/cooldown/dedup logic in both. If you fix a bug in one, the other diverges. Likely the Netlify one is dead code now — should be removed or made the single source of truth.

6. No graceful shutdown handler.

index.js — when Railway sends SIGTERM (deploy, scale-down), the process dies immediately. In-flight Firestore writes get cancelled, WebSockets aren't cleanly closed. Should add:

process.on("SIGTERM", async () => {

console.log("Shutting down gracefully...");

// Stop accepting new work, drain in-flight, close WS

for (const [, bot] of bots) bot.stop();

process.exit(0);

});

7. WebSocket reconnect uses fixed 5-second delay.

streamer-bot.js:131 — if Pusher is having an outage, all bots hammer it every 5 seconds. Should use exponential backoff with jitter (5s → 10s → 20s → 40s, cap at 60s).

8. _seenMsgIds size management is O(n) per message.

streamer-bot.js:113-119 — when the Set hits 500, it iterates to find the first key and deletes it. Per-message overhead. A circular buffer or LRU would be O(1).

9. In-memory giveaways cache in commands/giveaway.js is never cleaned up when a streamer is removed. Memory leak on streamer churn.

10. Health check endpoint is a fixed 200 OK.

discord-webhook.js:272 — req.url !== "/discord" always returns "WenBot OK". Real health check should verify Firestore connectivity + at least one bot is connected to Kick. Otherwise Railway thinks WenBot is healthy when it's actually broken.

11. No metrics or observability.

Zero counters, no error rate tracking, no request latency, no Pusher reconnect rate. For a SaaS, this is flying blind. Could add basic Prometheus-style counters or push to a free tier on Better Stack / Axiom.

12. kick-client.getChannelInfo uses an unofficial Kick endpoint (kick.com/api/v2/channels/) with no rate limit handling. If Kick changes or rate limits this, bot startup breaks silently. Should use the official api.kick.com/public/v1 endpoints where available, or at least handle 429s.

13. Pusher key is hardcoded in source.

streamer-bot.js:19 — fine since Pusher keys are public, but should be in env vars in case it ever rotates.

■ Updated cross-system findings

14. The Firestore service account in WenBotServer has full admin access.

firebase.js:18-21 — admin.initializeApp({ credential: admin.credential.cert(serviceAccount) }) gives the bot god-mode. This is correct (it needs to write across all streamer subcollections) but it means a single env var leak = total compromise. Should be tightly access-controlled in Railway and rotated periodically.

15. The bot/Netlify split duplicates ALL the Kick-side identity checks.

slot-request-add.js validates eligibility from the Netlify side

commands/slot-request.js validates the same thing from the bot side

Both are reading the same Firestore data and applying the same rules. Pick one. If it's the bot, delete the Netlify function (or make it a thin proxy that posts to a bot endpoint with HMAC).

■ Architectural take

The architecture has two duplicated code paths trying to do the same thing:

Bot path: Chat command → WenBotServer reads/writes Firestore directly using admin SDK

Netlify path: Web action → Netlify function reads/writes Firestore using admin SDK

The cleanest fix is to make WenBotServer the single source of truth for chat-driven actions and have Netlify functions only handle web-driven actions (dashboard, signup, Stripe). That eliminates the dedup problem at #5 and #15. The bot already has the live chat connection — it should own those flows entirely.

■ Updated priority list

If you only do five things:

Fix the Discord linking vulnerability (#1 above) — this is actively exploitable today and would let someone steal points from active viewers

Add firestore.rules to the repo and confirm tight rules in console — without this, every other security control is bypassable

HMAC auth between WenBotServer and Netlify functions — even if the rules are tight, the bot endpoints are still wide open today

Restrict bot restart in bot-manager.js to only relevant field changes — this is a stability issue causing chat message drops

Tie kickChannel to authenticated kickUserId — prevents channel hijacking

Then deal with the rest as time allows. The Discord linking issue specifically is one I'd patch tonight.

Want me to start on any of these?

here are the current firestore rules if this changes anything "rules_version = '2';

service cloud.firestore {

match /databases/{database}/documents {

match /streamers/{uid} {

allow read: if request.auth != null && request.auth.uid == uid;

// On CREATE — block any billing fields from being set at all

allow create: if request.auth != null && request.auth.uid == uid

&& !request.resource.data.keys().hasAny([

'plan',

'stripeSubscriptionActive',

'stripeCustomerId',

'stripeSubscriptionId',

'stripeActivatedAt',

'stripePaymentFailed'

]);

// On UPDATE — block billing fields from being changed

allow update: if request.auth != null && request.auth.uid == uid

&& !request.resource.data.diff(resource.data).affectedKeys().hasAny([

'plan',

'stripeSubscriptionActive',

'stripeCustomerId',

'stripeSubscriptionId',

'stripeActivatedAt',

'stripePaymentFailed'

]);

match /{subcollection}/{doc} {

allow read, write: if request.auth != null && request.auth.uid == uid;

}

}

match /system/{doc} {

allow read, write: if false;

}

}

}"

These rules are actually solid — much better than I feared. Let me update my review:

■ What the rules fix from my previous review

Critical #1 (no rules) → resolved. They're there, and they're reasonable:

Streamer docs are scoped to auth.uid == uid — viewers can't read other streamers' data

Billing fields (plan, stripeSubscriptionActive, etc.) are protected on both create and update — a user can't self-promote to Elite from the browser

system/* is completely denied — WenBot's master Kick tokens are safe from client reads

Everything else is denied by default — viewer pages can't bypass Netlify functions to read Firestore directly

Critical #4 (Kick tokens in Firestore) → partially resolved.

The streamer's own kickAccessToken is only readable by themselves now. So this becomes a "compromised account = compromised Kick tokens" issue, not a "anyone-on-internet" issue. Much smaller blast radius. Still worth moving to an encrypted-at-rest pattern eventually, but no longer urgent.

■ New concerns the rules introduce

1. The subcollection wildcard is too permissive.

This rule:

match /{subcollection}/{doc} {

allow read, write: if request.auth != null && request.auth.uid == uid;

}

…means the streamer can write ANYTHING to ANY subcollection under their own doc. Including:

Forging verified_users docs with underAffiliate: true (bypasses casino API check)

Inflating viewers/{x}/points to anything

Backdating winners and raffle_history

Pre-claiming bot_locks (could break the bot's command dedup)

Forging discord_links to claim other Kick usernames (this is the same vulnerability as #1 below, just via a different route)

For the SaaS model, most of these are self-harm (the streamer cheating themselves), but underAffiliate: true matters for the casino partner relationship. If a streamer is claiming inflated wagers under Gambulls's affiliate code by forging verified users, that's casino fraud, not just self-harm.

Recommended fix — replace the wildcard with explicit per-subcollection rules:

// Most subcollections: full control

match /{subcollection}/{doc} {

allow read, write: if request.auth.uid == uid;

}

// verified_users: write-restricted (server-only)

match /verified_users/{doc} {

allow read: if request.auth.uid == uid;

allow delete: if request.auth.uid == uid; // can remove

allow create, update: if false; // only admin SDK can verify

}

// discord_links: read-only on client

match /discord_links/{doc} {

allow read: if request.auth.uid == uid;

allow write: if false;

}

■ Critical issues that the rules DON'T fix

These all live in your serverless functions (which use admin SDK and bypass rules), so the rules can't help here:

A. Discord linking vulnerability (highest priority)

Still completely valid. verify-affiliate.js:156 accepts unverified kickUsername in the Discord dtoken flow. Anyone in the Discord server can claim to be any Kick user. This is exploitable today.

B. No HMAC between WenBotServer and Netlify functions

slot-request-add.js takes anonymous POSTs with kickUsername/slotName from any source. Need a shared WENBOT_HMAC_SECRET and signature verification.

C. kickChannel ownership not enforced

A user can set their kickChannel to "trainwreckstv" in setup.html. The rules allow the write (it's their own doc). Then any verify/tournament/bb-vote flow finding the streamer by kickChannel would find them. Need server-side check that kickChannel is tied to authenticated kickUserId.

D. WenBotServer restarts on every profile change

bot-manager.js:32-36 — stability bug, not security, but causes chat message drops.

■ Revised priority list

The Firestore rules being in place removes the panic-level concerns. The list is now:

Fix Discord linking vulnerability — actively exploitable, would let anyone in Discord steal a verified Kick user's points

Tighten the subcollection wildcard to deny client writes to verified_users and discord_links — closes the casino fraud / fake-link routes

HMAC the bot-to-Netlify endpoints — closes anonymous abuse

Tie kickChannel to kickUserId at the server level — closes hijacking

Fix bot-manager restart-on-every-change — stability

The first three are the security wins. #4 and #5 are quality improvements. Want me to start on any of them?